Product Requirements Document — Commitment Tracker

PDF generated 02 October 2026 from prd.md · app version 0.7.2

Last updated: 2026-10-02 · **App version:** 0.7.2 · Change-by-change history: CHANGELOG.md

Problem

Meetings generate spoken commitments ("I'll send that by Friday") that routinely get lost — no one wrote them down, no one owns following up, and there's no single place to see what's pending, in progress, or done across meetings. Existing task trackers (Jira, Sheets, etc.) require someone to manually transcribe the commitment into them, which mostly doesn't happen.

Product concept

One local web app (Flask + SQLite) that covers the whole loop:

1. **Capture** — a transcript arrives by upload or by a meeting tool's webhook (Fathom today).
2. **Extract** — commitments (task, owner, due date, source quote) are pulled out automatically, by the built-in ML model or by an AI service the customer already pays for.
3. **Assign** — each owner name is matched to a real person (email) from an uploaded people directory.
4. **Track & nudge** — status lives in the app and, optionally, in the team's Google Sheet (both directions); owners get reminders by email, Slack, Teams or a custom webhook.

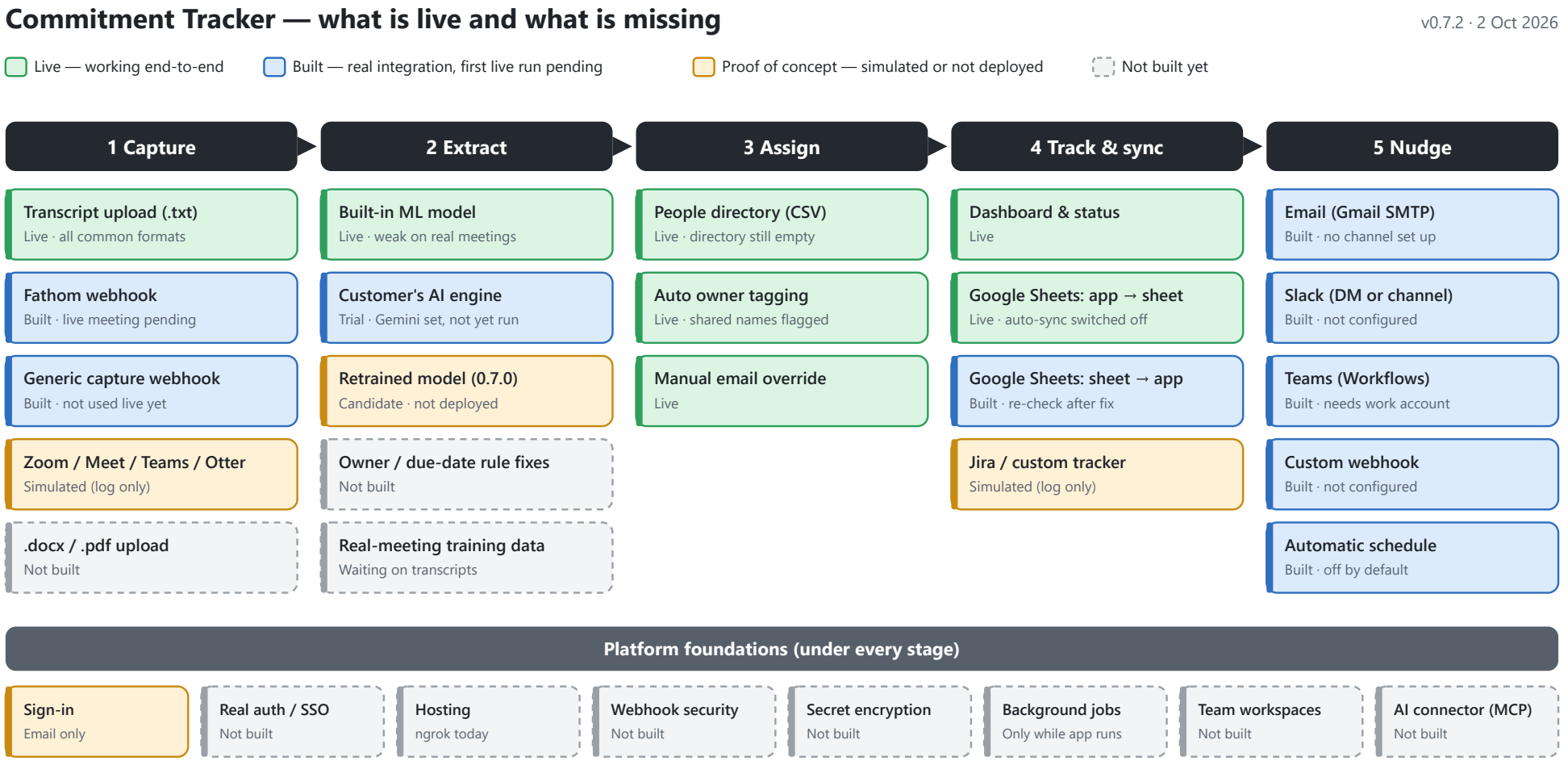
The product stays **free for everyone** — no subscription or paywall language anywhere. The earlier "freemium connector inside ChatGPT/Claude" go-to-market idea remains on hold.

Target user

Anyone who leaves meetings with verbal commitments that need follow-up — individually or in a small team — regardless of meeting platform or task tracker. Not gated behind an enterprise tier of any tool; the current test account is a personal Gmail with Zoom Basic, which shaped several choices below.

Current status

Status at a glance



"Built" items work against test stand-ins; each needs one real run on the test account to turn green. Details: "Current status" section of the PRD.

Read left to right as the product loop. Green = live and working end-to-end; blue = built with a real integration but not yet run against the real service on the test account; amber = proof of concept (simulated, or built but not deployed); grey dashed = not built. Totals: 7 live, 9 built and awaiting a first live run, 4 proof of concept, 10 not built. Regenerate with `python docs/make_status_diagram.py` when a status changes.

Legend for the tables below — **Verified live:** exercised against the real external service. **Verified with stand-ins:** the real code path was tested against local stand-ins for the external service (fake SMTP server, local webhook receivers, stubbed Google API), not the service itself.

1. Built, with a real integration

Capability	How it works	Verified	What it needs to use
Transcript upload	Paste or `.txt` file; binary files (e.g. `.docx`) rejected with a clear message	Live	Nothing
Transcript formats	`Name: text`, `[10:00 AM] Name:` header lines, `[09:00] Name: text`, WebVTT/SRT captions, curly quotes, "ACTION ITEMS" sections	Live (team's files)	Nothing
Commitment extraction — built-in model	Sentence-embedding classifier + owner/task/due-date rules, runs locally	Live	ML libraries + trained model (installed on this machine). **Quality is limited — see Data & ML**
Commitment extraction — customer's AI	Per-account engine: Claude (Anthropic SDK) or any OpenAI-compatible API (OpenAI, Azure OpenAI, Gemini, Groq, Ollama); falls back to built-in on any failure; side-by-side comparison + scoreboard	Claude: wiring verified, live call only with an invalid key. OpenAI-compatible: stand-in + Ollama cloud model live on a sample. **Gemini 2.5 Pro is configured on the test account but no meeting has run through it yet**	Customer's API key (chat subscriptions don't include API access). Optional feature with an off switch (`AI_EXTRACTION_ENABLED=0`) and a revert script — built on trial pending team review
Fathom → app (live transcripts)	Signed webhook (`/api/webhooks/fathom/<token>`, HMAC-verified); speaker turns used directly	Stand-ins (synthetic signed payload) — **not yet with a live Fathom meeting**	Free Fathom account, its bot in the meeting, ngrok (installed) or hosting, webhook secret saved in Integrations
Generic transcript webhook	`/api/webhooks/transcript-capture/<token>` for any tool that can POST JSON	Stand-ins	Public URL (ngrok)
Google Sheets — app → sheet	Real Sheets API; rows written to columns A:E with a header; auto-sync of new commitments and status changes; bulk "Sync all to sheet"	Live (test account's sheet, rows written); fixes in 0.3.1/0.4.0 verified with stand-ins	One-time Google login (done on this machine). **Auto-sync is currently OFF on the test account** — re-save the tracker settings to turn it on
Google Sheets — sheet → app	Apps Script `onEdit` trigger → `/api/webhooks/tracker-update`; accepts Done/Closed/In progress/Open…	Stand-ins after the 0.3.1 fix; **not yet re-confirmed from the real sheet**	Apps Script with installable trigger, current ngrok URL
People directory & owner tagging	CSV upload (First/Last Name, Email ID, Corporate User Name); every new task auto-tagged; shared names flagged, never guessed; manual override	Stand-ins + test DB	Upload a people CSV (**directory is empty on the test account**)
Nudges — email	SMTP (Gmail app password, Microsoft 365, SendGrid…) to the owner's email	Stand-in SMTP server — **not yet sent through real Gmail**	SMTP details; no channel configured yet
Nudges — Slack	Bot token → direct message (looked up by email), or webhook → channel	Stand-ins	Free Slack app; not configured
Nudges — Teams	Power Automate Workflows webhook (channel post, or DM via a custom flow)	Stand-ins	**Microsoft 365 work/school account** (personal Teams can't)
Nudges — custom	JSON POST of recipient/task/due/status to any URL	Stand-ins	A URL

Capability	How it works	Verified	What it needs to use
Automatic nudges	Every 15 min while the app runs: day-before / on-due / daily-overdue, no repeats, finished tasks skipped	Stand-ins	Off by default; switch on in Integrations
AI-suggested setup steps for custom apps	Claude call when `ANTHROPIC_API_KEY` is set, template otherwise	Template path live	Optional API key

2. Built as proof of concept — no real integration

Capability	What exists	What's missing
Jira tracker	Settings form, mocked "would create issue" outbound log; inbound status webhook is real code	Real Jira API calls; never tested with a real Jira automation
Custom tracker	Settings form, mocked outbound	Any real call
Zoom / Google Meet / Teams / Otter as transcript sources	Settings forms, setup steps, mocked "Check now"	Real APIs. Not usable on the test account anyway: Zoom Basic has no cloud transcripts, Meet's transcript API needs Google Workspace, Otter's API is Enterprise-only — Fathom is the working route
Retrained model	`ml/action_detector_candidate.keras` (fewer false positives, more misses)	Not deployed — regressed on the only business meeting (see Data & ML)
Sign-in	Email address only, no password	Real authentication
Demo / simulate buttons	Demo meetings, "Simulate incoming transcript", "Simulate update from tracker"	— (test aids, by design)

3. Not built

- Real authentication (OAuth/SSO) and multi-user team workspaces — every meeting belongs to one email.
- Hosting — inbound webhooks only reach the app through ngrok, whose free URL changes on restart.
- Security hardening: webhook authentication for `/api/webhooks/tracker-update` (trusts email + row reference), encryption of stored secrets (API keys, SMTP password, Slack token are plain text in the local DB).
- `.docx` / `.pdf` transcript upload in the app (the training pipeline can read `.docx`; the upload form takes `.txt` only).
- An AI-assistant connector (MCP server / GPT Action).
- A real background job runner — automatic nudges only run while the app is running on this machine.
- Mobile / native clients.

Data & ML status

- **Built-in model** was trained only on synthetic, templated meetings (99.99% on that data, which says little). On the real-format FY2027 meeting it finds 10 of 13 commitments but flags 25 false ones.
- **0.7.0 retraining** used the team's transcripts, which turned out to be ~5 independent sources: two templates repeated hundreds of times, two AI-rewritten workshops, one real recording that is a class lecture — and **no real business meetings**. Leave-one-family-out results: average F1 0.24 → 0.44, but FY2027 0.42 → 0.38 (finds 4 of 13). Not deployed.
- **Labels** for 2,996 distinct lines (158 commitments) are in `training_data/labelled_utterances.csv` with borderline calls noted — **awaiting team review**.
- **Owner and due-date extraction** is rule-based and misses common shapes: read-out assignment lists ("Kevin Heys (Due: September 27): …") are credited to the reader, and "Due: September 26" / "end of this month" aren't parsed.
- **Next data needed:** 5–10 real business-meeting transcripts (Fathom recordings are the easiest source) — the team is collecting these.

Source control

- GitHub (`origin`): 3 commits, last 2026-08-30. Local `main` has one more commit (2026-09-03, inbound transcript webhook) that **has not been pushed**.
- Everything from 0.2.0 onward (~20 changed or new files) is **uncommitted local work**, by decision — see CHANGELOG.md, which also records snapshots for reverting trial features.
- Excluded from git on purpose: the local database, Google credentials (`client_secret_*.json`, `google_token.json`), embedding cache, `_snapshots/`.
- Not excluded and needing a decision before any commit: training CSVs (~21 MB), the team's transcript folder (~7 MB), FY2027 transcript files (real meeting content), the original Colab script.

Key user stories

1. As someone who just left a meeting, I want the commitments that came out of it in one place, without typing them in.
2. As a commitment owner, I want to mark my task pending / in progress / done, and see whether it's overdue.
3. As a user whose team lives in a Google Sheet, I want commitments to appear there automatically and status changes to flow both ways.
4. As an owner, I want a reminder before my deadline, on the channel my team already uses.
5. As the person running the meeting, I want each task tagged with the right person's email automatically — and to fix it when two people share a name.
6. As a company with an AI subscription, I want to use that AI for extraction and compare it with the built-in model before switching.
7. As a first-time visitor, I want to try the app without creating an account or handing over a password.
8. As someone picking an unfamiliar setting (a Slack token, an SMTP app password), I want the app to tell me where to get it.

Open questions / decisions

- **Model:** keep the current built-in model or switch to the 0.7.0 candidate until real meetings arrive? (Recommendation: keep current.)
- **Trial feature:** keep or revert the 0.5.0 AI-extraction feature after team review.
- **Labels:** does the team agree with the commitment definition used for labelling?
- **Git:** when to commit, and which of the large/sensitive files above belong in the repository.
- **Hosting**, and with it authentication for machine callers (webhooks, a future connector).
- **Connector platform** (MCP, GPT Action, both, neither) — still on hold.

Roadmap (proposed order)

1. ~~Local prototype: dashboard, tracker/nudge settings, chart~~ — done (0.1.0).
2. ~~Built-in commitment extraction~~ — done (0.2.0), quality work ongoing.
3. ~~Real Fathom capture and two-way Google Sheets sync, auto-sync~~ — done (0.3.0–0.4.0).
4. ~~Optional customer-AI extraction with comparison~~ — done on trial (0.5.0).
5. ~~People directory, owner tagging, real nudges~~ — done (0.6.0).
6. ~~Retraining pipeline and parsing fixes~~ — done (0.7.0); model not yet improved enough to deploy.
7. **Next, no new data needed:** rule fixes for assignment lists and due-date phrases; `.docx` upload; webhook authentication and secret encryption.
8. **Next, with real transcripts:** label (with team review), retrain, evaluate on held-out real meetings, decide deployment.
9. Go-live checks on the test account: one real Fathom meeting, one real Gmail nudge, a sheet → app status change.
10. Hosting decision → real authentication → connector.